

Laboratorium
Robotika dan Sistem Cerdas
Departemen Teknik Komputer
Institut Teknologi Sepuluh Nopember

Laporan Sementara
Praktikum Sistem Mikroprosesor dan Sistem Tertanam

Pengenalan Mikroprosesor

Athallah Ramadhan - 5024241074

8 November 2026

1 Pendahuluan

1.1 Latar Belakang

Mikrokontroler adalah komponen inti pada sistem tertanam (*embedded system*) modern, mulai dari perangkat rumah tangga dan sistem kendali industri hingga perangkat IoT. Mikrokontroler menggabungkan prosesor, memori, dan *peripheral* dalam satu chip, sehingga sebuah sistem dapat membaca kondisi lingkungan, memprosesnya, lalu mengendalikan keluaran secara otomatis. Bagi mahasiswa Teknik Komputer, pemahaman dasar tentang cara memprogram dan mengakses perangkat keras mikrokontroler menjadi fondasi bagi materi lanjutan seperti sensor, aktuator, komunikasi data, dan sistem waktu nyata.

Pada praktikum ini digunakan dua mikrokontroler yang sering digunakan. STM32 Blackpill berbasis ARM Cortex-M4 dan diprogram melalui ST-Link dengan protokol SWD. ESP32 berinti ganda dengan WiFi dan Bluetooth terintegrasi, dan dapat diprogram langsung melalui USB. Keduanya juga dapat diprogram dengan tingkat abstraksi yang berbeda. Framework Arduino menyederhanakan program menjadi `setup()` dan `loop()` sehingga mudah dipelajari. ESP-IDF adalah framework *native* dari Espressif yang berjalan di atas FreeRTOS, dengan struktur `app_main()` dan akses langsung ke *task*, *semaphore*, dan *queue*.

Setelah *environment* siap digunakan, hal pertama yang perlu dikuasai adalah pengendalian GPIO (*General Purpose Input/Output*). Sebuah pin GPIO dapat dipakai dalam tiga mode. Mode pertama adalah *digital output*, misalnya menyalakan LED pada program Blink. Mode kedua adalah *digital input*, misalnya membaca tombol. Mode ketiga adalah *analog input* melalui ADC. Mode *digital input* membawa persoalan yang tidak tampak pada *output*. Pin yang tidak terhubung ke level tegangan apa pun berada dalam kondisi *floating*, sehingga nilainya tidak pasti dan dapat berubah acak akibat *noise*. Persoalan ini diatasi dengan resistor *pull-up* atau *pull-down*, baik eksternal maupun internal. Praktikan perlu memahami perbedaan keduanya, termasuk keterbatasan bahwa tidak semua pin ESP32 punya resistor *pull* internal.

1.2 Dasar Teori

Mikrokontroler seperti STM32 Blackpill (ARM Cortex-M4) dan ESP32 (*dual-core* Xtensa LX6, dengan WiFi dan Bluetooth) diprogram lewat PlatformIO di VSCode. Blackpill memakai framework Arduino (STM32duino) dan diprogram lewat ST-Link via SWD, sedangkan ESP32 bisa memakai framework Arduino (`setup()` dan `loop()`) atau ESP-IDF yang berbasis FreeRTOS dengan titik masuk `app_main()`.

Pin GPIO yang membaca tombol akan *floating* jika tidak ditarik ke level tertentu, sehingga nilainya acak karena *noise*. Resistor *pull-up* menarik pin ke 3.3 V, jadi pin HIGH saat tombol dilepas dan LOW saat ditekan. *Pull-down* menarik pin ke GND, sehingga logikanya kebalikan. Keduanya bisa dipasang eksternal (misalnya 10 kΩ) atau diaktifkan internal lewat `INPUT_PULLUP` dan `INPUT_PULLDOWN`, kecuali pin *input-only* GPIO 34–39 yang tidak punya resistor internal.

Kontak mekanik tombol menghasilkan transisi HIGH-LOW berulang dalam waktu singkat (*bouncing*), sehingga satu tekanan bisa terbaca beberapa kali. *Debouncing* mengatasinya dengan menunggu sinyal stabil sekitar 20–50 ms sebelum dianggap valid.

Selain *digital output* dan *input*, GPIO bisa dipakai sebagai *analog input* lewat ADC. ESP32 punya ADC 12-bit (0–4095) dengan rentang 0–3.3 V. LDR dirangkai sebagai pembagi tegangan, dan

tegangan hasil konversinya:

$$V_o = V_{in} \frac{R_1}{R_{LDR} + R_1} \tag{1}$$

$$V = \text{raw} \times \frac{3.3}{4095} \tag{2}$$

2 Tugas Pendahuluan

Bagian ini berisi jawaban dari tugas pendahuluan yang telah Anda kerjakan, beserta penjelasan dari jawaban tersebut.

1. Framework Arduino menyederhanakan program menjadi `setup()` dan `loop()` dengan fungsi yang mudah seperti `pinMode()` dan `delay()`, sehingga cepat dipelajari, tetapi akses ke fitur *low-level* dan RTOS terbatas. ESP-IDF adalah framework *native* Espressif yang berbasis FreeRTOS dengan titik masuk `app_main()`, sehingga praktikan bisa langsung memakai *task*, *semaphore*, dan *queue* serta mengatur fitur seperti WiFi secara mendalam. Kekurangannya, ESP-IDF lebih rumit dan kurva belajarnya lebih curam.

2. STM32 Blackpill tidak memiliki programmer maupun chip USB-to-Serial onboard, sehingga *firmware* harus dimasukkan lewat ST-Link eksternal melalui protokol SWD. Pada ESP32 DevKit sudah terpasang chip USB-to-Serial (CP2102 atau CH340) yang mengubah sinyal USB dari PC menjadi UART, dan *bootloader* bawaan ESP32 menerima *firmware* lewat UART tersebut. Karena itu ESP32 bisa diprogram langsung lewat kabel USB tanpa programmer tambahan.

3. Pada konfigurasi *pull-up*, pin ditarik ke 3.3 V lewat resistor, sehingga pin terbaca HIGH saat tombol tidak ditekan dan LOW saat ditekan (pin terhubung ke GND). Pada konfigurasi *pull-down*, pin ditarik ke GND lewat resistor, sehingga pin terbaca LOW saat tombol tidak ditekan dan HIGH saat ditekan (pin terhubung ke 3.3 V). Jadi logika keduanya saling berkebalikan.

4. *Bouncing* adalah ketika sinyal tombol yang berubah HIGH-LOW berulang dalam waktu sangat singkat saat tombol ditekan atau dilepas. Hal ini terjadi karena kontak logam di dalam tombol mekanik tidak langsung diam(0 ke 1), melainkan memantul dan bergetar sebelum benar-benar stabil, sehingga satu tekanan bisa terbaca beberapa kali. Pendekatan untuk mengatasinya antara lain *debouncing* lewat *software*, yaitu menunggu sinyal stabil selama 20–50 ms (misalnya dengan `millis()`) sebelum dianggap valid, dan *debouncing* lewat *hardware*, yaitu memasang kapasitor sebagai filter RC pada tombol, dengan tambahan *Schmitt trigger* bila perlu.

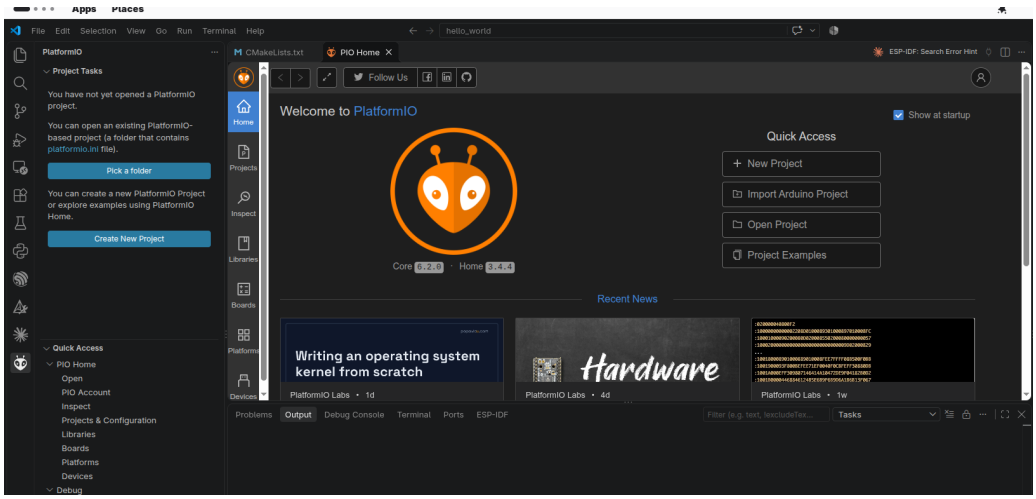
5. ADC mengubah sinyal analog menjadi digital lewat tiga tahap: *sampling* (mengambil nilai tegangan pada waktu tertentu), kuantisasi (membulatkan tegangan ke salah satu level diskret), dan *encoding* (menyatakan level itu sebagai bilangan biner). Resolusi bit menentukan jumlah level dan ukuran langkah tegangan terkecil yang bisa dibedakan. Pada ESP32 dengan referensi 3.3 V:

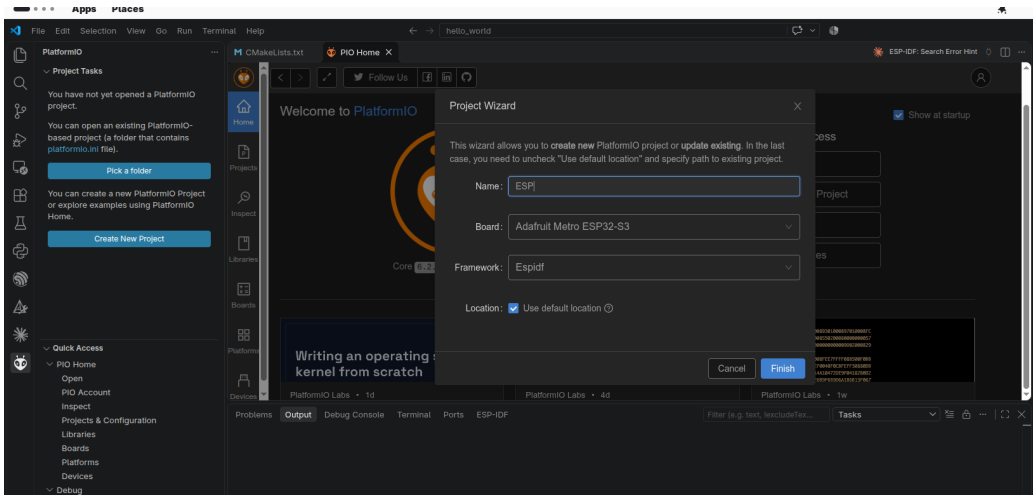
$$\text{12-bit: } 2^{12} = 4096 \text{ level}, \quad \frac{3.3}{4095} \approx 0.8 \text{ mV}$$

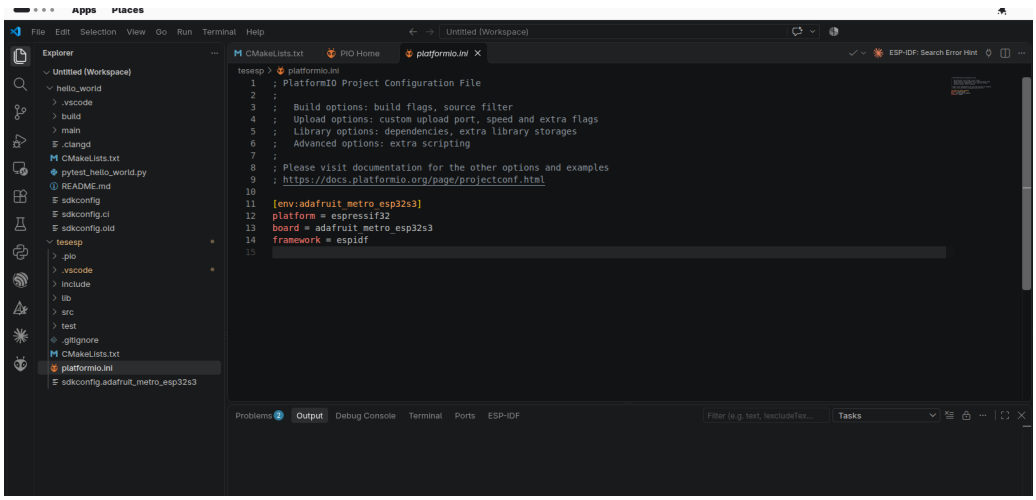
$$\text{10-bit: } 2^{10} = 1024 \text{ level}, \quad \frac{3.3}{1023} \approx 3.2 \text{ mV}$$

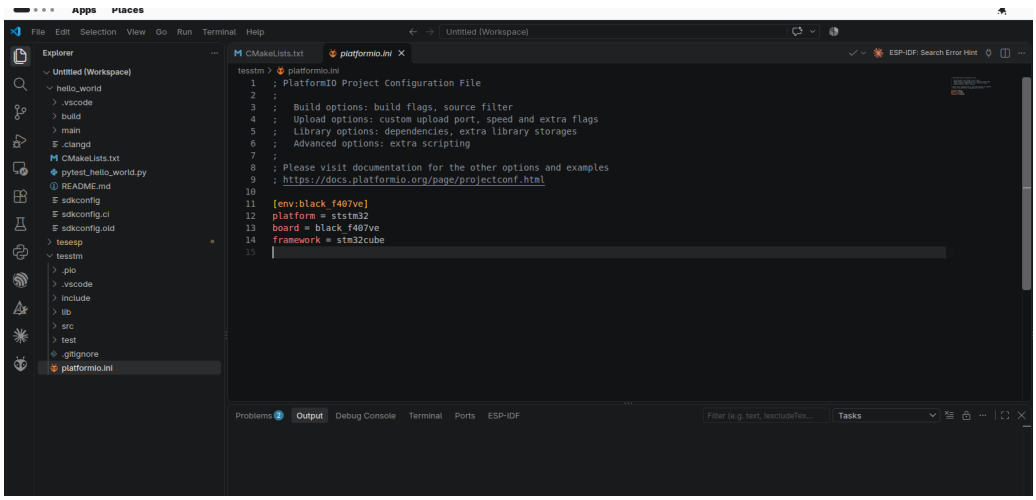
Semakin besar resolusi bit, semakin kecil langkah tegangannya sehingga hasil pembacaan semakin presisi.

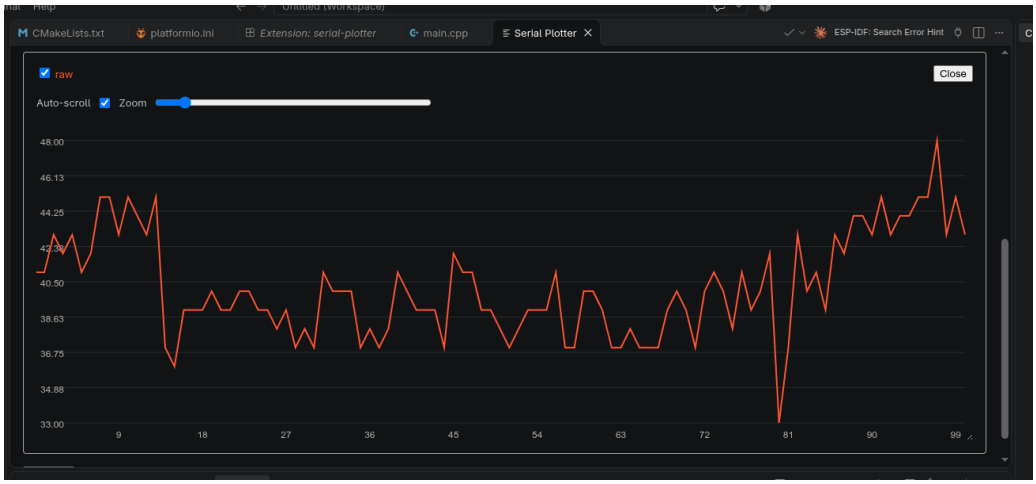
6. 1 - Platform IO, 2,3 - ESP, 4-STM32,5-SerialPlotter








Apps Places
File Edit Selection View Go Run Terminal Help
Untitled (Workspace)
Explorer
Untitled (Workspace)
hello_world
.vscode
build
main
.clangd
CMakeLists.txt
pytest_hello_world.py
README.md
sdkconfig
sdkconfig.ci
sdkconfig.old
tesesp
tesstm
.pio
.vscode
include
lib
src
test
.gitignore
platformio.ini
CMakeLists.txt
platformio.ini
ESP-IDF: Search Error Hint
tesstm > platformio.ini
; PlatformIO Project Configuration File
;
; Build options: build flags, source filter
; Upload options: custom upload port, speed and extra flags
; Library options: dependencies, extra library storages
; Advanced options: extra scripting
;
; Please visit documentation for the other options and examples
; https://docs.platformio.org/page/projectconf.html
[env:black_f407ve]
platform = ststm32
board = black_f407ve
framework = stm32cube
Problems 2
Output
Debug Console
Terminal
Ports
ESP-IDF
Tasks


CMakeLists.txt
platformio.ini
Extension: serial-plotter
main.cpp
Serial Plotter
ESP-IDF: Search Error Hint
raw
Close
Auto-scroll
Zoom
48.00
46.13
44.25
42.38
40.50
38.63
36.75
34.88
33.00
9
18
27
36
45
54
63
72
81
90
99